Docker技术应用与实战

一、 Docker的安装和介绍

1 没有Docker的世界

1.1 场景一 节省项目环境部署时间

> 问题：每次部署项目到测试、生产等环境，都要部署一大推以来的软件，工具，花费很长时间。

使用docker可以实现一次构建多次部署，一劳永逸。

1.2 场景二— 环境一致性

> 问题：程序在我这里跑跑得好好的，在你那怎么就不行呢？

版本一致性：包括操作系统、数据库、中间件的版本，被测系统的版本。

配置一致性：系统（操作系统/数据库、中间件/被测试系统）参数的配置一致。

1.3 场景三—持续集成(devops)

> 问题：老板突然说5分钟后在服务器上演示一下项目的最新成果，你能做到嘛？

1.4 场景四—微服务于容器

1.5 场景五—弹性伸缩

2 Docker是什么

> Docker 是一个开源的应用容器引擎，基于 Go 语言 并遵从 Apache2.0 协议开源。

> 官方团队对Logo的解释： **「The logo is a Whale carrying a stack of containers. It is on its way to deliver those to you」**

为什么是"集装箱技术"

> 定义: Docker是一个开源的应用容器引擎，开发者可以打包他们的应用及依赖到一个可移植的容器中，发布到流行的Linux机器上，也可实现虚拟化

Docker,其实就是可以打包程序和运行环境，把环境和程序一起发布的容器。

其他人到你的程序后可以直接运行，避免出现一次编译到处调试的尴尬局面~

主要为了解决:

”在我机器是正常的 为什么到你的机器就不正常了“ 的问题

3 容器和虚拟机

3.1 Docker与虚拟化

虚拟化技术定义

> Virtual Machine(又名VM) 虚拟化，是指通过虚拟化技术将一台计算机虚拟为多台逻辑计算机。在一台计算机上同时运行多个逻辑计算机，每个逻辑计算机可运行不同的操作系统，并且 应用程序都可以在相互独立的空间内运行而互不影响，从而显著提高计算机的工作效率。

简单的说，「虚拟化」，顾名思义，就是「虚拟」的，是「假」的，是形而上的，是观念上、逻辑上的划分。

> 举栗说明。假设你现在手头上有一套房子，并且你要把它给租出去，租金的市场价是2000元/月。那么有没有办法把它租得更贵，获得6000元的租金呢？你想到了两个办法：

1. 找一个不差钱的傻X租户，让他每月交6000元。
2. 把房子租给4个单身狗，租金为1500元/人/月。

3.2 Docker和虚拟机比喻

什么是物理机

什么是虚拟机

什么是Docker

3.3 总结

4 Docker概念

Docker技术的三大核心概念，分别是：

- 镜像（Image）
- 容器 (Container)
- 仓库 (Repository)

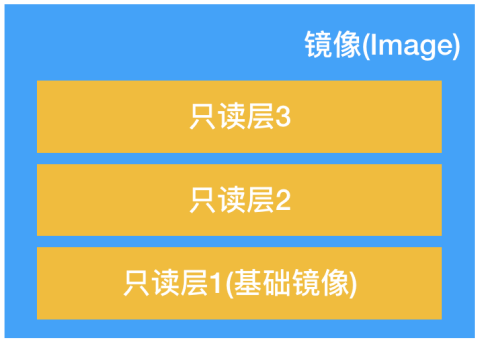
为了更好的理解，先讲大家讲个小故事:

听完了故事，咱们再来理解docker的两句口号就更好理解了

- **「Build, Ship and Run (搭建，发送，运行)」**
- **「Build once，Run anywhere (搭建一次，运行到任何地方)」**

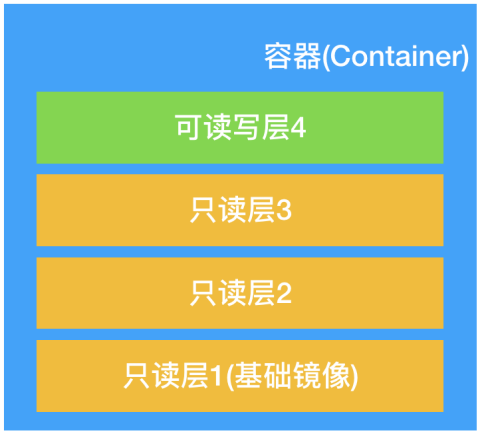
镜像

> 我们可以将 Docker 镜像理解为包含应用程序以及其相关依赖的一个基础文件系统(存储着我们的程序和数据文件等等信息，镜像层可以理解为基本的镜像，可以通过组合多个镜像层生成一个新的镜像；也可以将一个镜像可以作为一个镜像层，成为组合别的镜像的一份子)，在 Docker 容器启动的过程中，它以只读的方式被用于创建容器的运行环境。

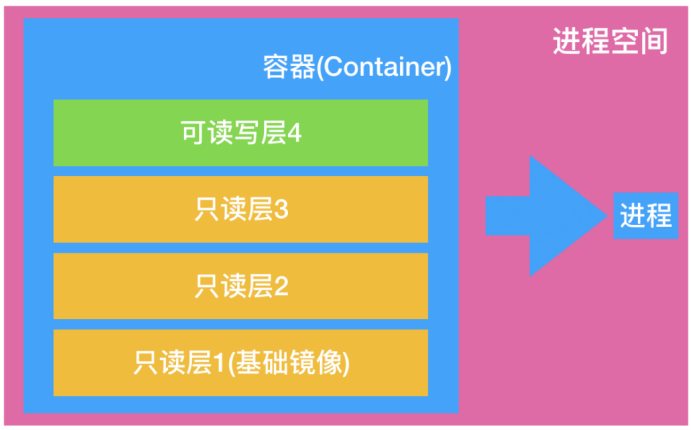


容器

> 容器和镜像一样，也是若干层的叠加，唯一区别是所有只读层的最上面一层，是一层可读可写层，可以记住这个简单的公式：容器 = 容器镜像 + 可读可写层



> 前面说镜像的时候我们有提到它是一个静态文件系统，那么既然容器跟它一样，所以，镜像跟容器都是一个静态概念，但是，容器是可以run起来的，当容器run起来时，就会成为一个被称作运行态容器的东西，如下图所示



运行态容器(run container)是由静态容器(可读写的静态文件系统)、一个隔离的进程空间、以及运行于其中的程序进程所组成。我们通常用docker run运行起来的其实就是运行态容器，而使用docker stop停止后的容器，就是静态容器。

Docker改变了什么?
-> 面向产品：产品交付
-> 面向开发：简化环境配置
-> 面向测试：多版本测试
-> 面向运维：环境一致性
-> 面向架构：自动化扩容

5 Docker安装

系统环境

序号	软件	版本
1	Centos	8
2	Linux内核	3.8之上

5.1 关闭防火墙

```
systemctl stop firewalld.service
```

5.2 关闭selinux

```
vim /etc/selinux/config
SELINUX=disabled
```

5.3 安装依赖包

```
yum install -y yum-utils device-mapper-persistent-data lvm2
```

5.4 安装aliyun镜像下载

```
yum-config-manager  --add-repo https://download.docker.com/linux/centos/docker-ce.repo
```

5.5 安装docker引擎

```
yum install docker-ce docker-ce-cli containerd.io


报错
rpm -q podman
dnf remove podman
yum erase podman(容器管理工具) buildah （容器镜像）
```

5.6 启动docker

```
systemctl start docker
```

5.7 设置docker自启动

```
systemctl enable docker
```

5.8 测试docker是否安装成功

```
docker run hello-world
```

5.9 配置阿里云镜像加速

```
sudo mkdir -p /etc/docker
sudo tee /etc/docker/daemon.json <<-'EOF'
{
"registry-mirrors": ["https://vmdgahdo.mirror.aliyuncs.com"]
}
EOF
sudo systemctl daemon-reload
sudo systemctl restart docker
```

二、 Docker镜像和容器

1 帮助命令

```
# 显示docker的版本信息
docker version
# 显示docker 的系统信息，包括镜像和容器的数量
docker info
# 帮助命令
docker 命令 --help
```

2 镜像命令

> 在 Docker 的术语里，一个只读层被称为镜像，一个镜像是永久不会变的。

查看镜像

```
[root@docker ~]# docker images
REPOSITORY   TAG       IMAGE ID   CREATED   SIZE
```

镜像搜索

```
docker serach
```

镜像下载

```
docker pull 镜像名:[tag]
```

删除镜像

```
docker rmi 源名字
docker rmi image id
```

3 容器命令

> 当我们有了镜像，才可以创建容器, 下载一个centos镜像来学习

3.1 新建容器并启动

```
docker run [可选参数] image
# 常用参数
--name     #创建容器名字用来区分容器
-d         #后台交互式运行
-it        #使用交互方式运行，  进入容器查看内容
-p         #指定容器端口号  -p 8080:8080

           -p 主机端口:容器端口
-P         #随机指定端口号

#启动容器
```

3.2 列出容器

```
docker ps #列出当前运行的容器
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

3.3 退出

```
#如果docker容器在启动的时候没有加 -d 参数
exit # 会直接停止同期并退出
Ctrt + p + q # 容器不停止 并退出
```

3.4 删除容器

```
docker rm 容器id   # 删除指定容器  如果不能删除  强制删除  rm -r
```

3.5 启动与停止容器

```
docker start 容器id    # 启动容器
docker stop 容器id     # 停止容器
docker rm  容器id      # 删除容器
docker restart 容器id  # 重新启动容器
docker kill 容器id     # 强制性停止当前容器
```

4 其他命令

```
docker run -d 镜像名字
问题原因   容器使用后台运行，就必须要有一个前台进程。

你是老板  你聘用一个上班   这个人来公司不干活    就开除了。。。
```

4.1 查看日志

```
启动应用服务如果失败是不是就是看日志

 docker run -d centos /bin/bash -c "while true;do echo baizhan;sleep 1 ;done"
c607c6e88514d9e79f21c37bc5274af647a1fe1a8ce1b39aab04cb77b57f3167
[root@docker opt]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS
PORTS     NAMES
c607c6e88514   centos    "/bin/bash -c 'while…"   2 seconds ago   Up 2 seconds
          pedantic_taussig
[root@docker opt]#
[root@docker opt]#
[root@docker opt]# docker logs -tf --tail 10 c6
2021-07-14T03:14:28.729972504Z baizhan
2021-07-14T03:14:29.738033536Z baizhan
2021-07-14T03:14:30.742737466Z baizhan
2021-07-14T03:14:31.748372094Z baizhan
```

4.2 查看容器中的进行信息

```
docker top 容器id
```

4.3 查看镜像元数据

```
docker inspect  容器id
```

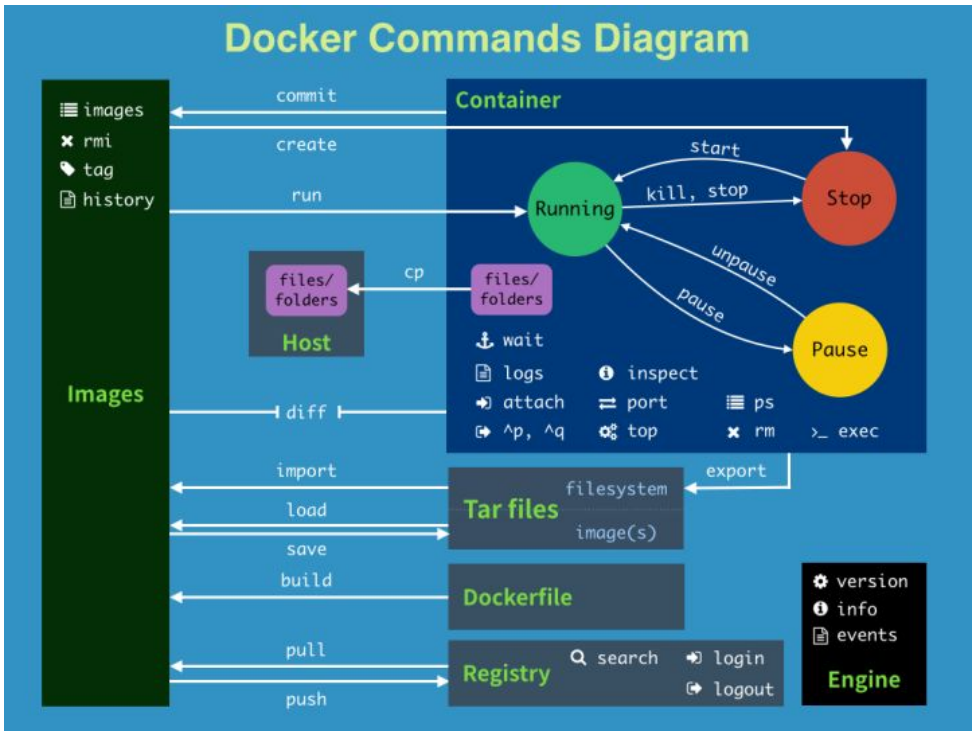
4.4 进入容器

```
方式一
docker exec -it /bin/bash 容器id   #开启一个新的终端，  常用
方法二
docker attach 容器id               # 进入容器正在执行的终端，  不会开启新的进程
```

4.5 从容器拷贝文件到主机上

```
docker cp 容器id：容器内路径     目录路径
```

5 小结



6 Docker实践

6.1 Docker安装Nginx

> Nginx

安装步骤

```
1. 查看nginx镜像
docker search nginx
2. 下载nginx镜像
docker pull nginx
3. 运行nginx镜像
docker run -d --name mynginx -p 3344:80 nginx
```

端口绑定:

```
-p 80:80
```

思考一个问题： 每次要修改nginx配置是不是都得去容器里面？

6.2 Docker安装Tomcat

> Tomcat

安装步骤

```
1. 查看tomcat 镜像
docker search tomcat
2. 下载tomcat镜像
docker pull tomcat
3. 运行tomcat镜像

docker run -it --rm tomcat:9.0

--rm   测试容器   我们这个容器运行完之后就自动删除
```

启动tomcat容器

```
[root@docker ~]# docker run -d --name mytomcat -p 3355:8080 tomcat:9.0
1a97194eba677b6bce69bf873222d5f746cc3ce00f03778bca3db0202a230c0a
[root@docker ~]# docker ps
CONTAINER ID   IMAGE        COMMAND                  CREATED          STATUS
     PORTS                                       NAMES
1a97194eba67   tomcat:9.0   "catalina.sh run"        11 seconds ago   Up 10
seconds   0.0.0.0:3355->8080/tcp, :::3355->8080/tcp   mytomcat
33b21d53f65d   nginx        "/docker-entrypoint.…"   24 minutes ago   Up 24
minutes   0.0.0.0:80->80/tcp, :::80->80/tcp           mynginx
[root@docker ~]# docker exec -it 1a /bin/bash
root@1a97194eba67:/usr/local/tomcat# ls
BUILDING.txt     LICENSE  README.md  RUNNING.txt  conf  logs        temp
webapps.dist
CONTRIBUTING.md  NOTICE   RELEASE-NOTES  bin          lib   native-jni-lib
webapps  work
root@1a97194eba67:/usr/local/tomcat# cd webapps
root@1a97194eba67:/usr/local/tomcat# ls
BUILDING.txt     LICENSE  README.md  RUNNING.txt  conf  logs        temp
webapps.dist
CONTRIBUTING.md  NOTICE   RELEASE-NOTES  bin          lib   native-jni-lib
webapps  work
root@1a97194eba67:/usr/local/tomcat# rm -rf webapps
root@1a97194eba67:/usr/local/tomcat# mv webapps.dist/ webapps
root@1a97194eba67:/usr/local/tomcat# exit
exit
[root@docker ~]# docker ps -a
CONTAINER ID   IMAGE        COMMAND                  CREATED          STATUS
     PORTS                                       NAMES
1a97194eba67   tomcat:9.0   "catalina.sh run"        2 minutes ago    Up 2
minutes    0.0.0.0:3355->8080/tcp, :::3355->8080/tcp   mytomcat
33b21d53f65d   nginx        "/docker-entrypoint.…"   26 minutes ago   Up 26
minutes   0.0.0.0:80->80/tcp, :::80->80/tcp           mynginx
[root@docker ~]# docker restart 1a
1a
```

6.3 Docker安装Mysql

> Mysql

安装步骤

```
1. 查看镜像
docker search mysql:5.7
2. 下载镜像
docker pull mysql:5.7
3. 启动镜像
docker run --name mysql -d  -p 3307:3306 -e MYSQL_ROOT_PASSWORD=123456
mysql:5.7

-p 我们以后使用3307 端口来访问mysql 为了防止黑客攻击

-e  配置环境变量
MYSQL_ROOT_PASSWORD 设置容器内mysql root  密码
```

6.4 Docker安装Redis

> Redis

安装步骤

```
1. 搜索镜像
docker search redis
2. 下载镜像
docker pull redis
3. 启动镜像
docker run --name my-redis  -d redis
```

三、Docker容器数据卷

1 什么是容器数据卷

> Docker中的数据可以存储在类似于虚拟机磁盘的介质中，在Docker中称为数据卷（Data Volume）

1.1 思考

- Docker容器删除后，在容器中产生的数据还在吗？
- Docker容器和外部机器可以直接交换文件吗？
- 容器之间能进行数据交互？

1.2 数据卷概念

- 数据卷是宿主机中的一个目录或文件。
- 当容器目录和数据卷目录绑定后，对方修改会立即同步。
- 一个数据卷可以同时被多个容器同时挂载。
- 一个容器也可以被挂载多个数据卷。

1.3 数据卷作用

- 容器数据持久化。
- 外部机器和容器间接通信。
- 容器之间数据交换。

一句话： 卷技术就是为了实现数据的持久化和同步操作，容器间也是数据共享。

1.4 配置数据卷

命令

```
docker run ..... -v 宿主机目录（文件）:容器内目录(文件)

注意事项
1. 目录必须是绝对路径
2. 如果目录不存在则会自动创建
3. 可以挂载多个数据卷
```

2 Volume基本使用

2.1 管理卷

```
# docker volume create edc-nginx-vol // 创建一个自定义容器卷
# docker volume ls // 查看所有容器卷
# docker volume inspect edc-nginx-vol // 查看指定容器卷详情信息
```

2.2 使用指定卷容器

> 有了自定义容器卷，我们可以创建一个使用这个数据卷的容器，这里我们以nginx为例：

```
docker run -d -it --name=edc-nginx -p 8800:80 -v edc-nginx-
vol:/usr/share/nginx/html nginx
```

其中，-v代表挂载数据卷，这里使用自定数据卷edc-nginx-vol，并且将数据卷挂载到/usr/share/nginx/html （这个目录是yum安装nginx的默认网页目录）。

如果没有通过-v指定，那么Docker会默认帮我们创建匿名数据卷进行映射和挂载。

3 实战Mysql数据卷

```
docker run -d -p 3306:3306 -v /opt/mysql/conf:/etc/mysql/conf.d  -v
/opt/mysql/data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=123456 --name my-mysql
mysql:5.7
```

4 具名和匿名挂载

```
匿名挂载：卷挂载只写容器里面的路径，不写容器外的路径

如何确定是匿名挂载还是具名挂载，还是指定路径挂载

-V     容器内路径                              #匿名挂载
-v     卷名:容器内路径                          #具名挂载
-v     /宿主内路径:容器内路径                    #指定路径挂载
```

拓展

```
通过 -v 容器内路径   ro   rw  改变目录读写权限
ro  readonly  # 只读
rw  readwrite # 可读可写

docker run -d -P --name nginx02 -v my-nginx:/etc/nginx:ro nginx
docker run -d -P --name nginx02 -v my-nginx:/etc/nginx:rw nginx
```

四、DockerFile完全指南

1 什么是DockerFile

1.1 概述

> Dockerfile是一个创建镜像所有命令的文本文件, 包含了一条条指令和说明, 每条指令构建一层, 通过docker build命令,根据Dockerfile的内容构建镜像,因此每一条指令的内容, 就是描述该层如何构建.有了Dockefile, 就可以制定自己的docker镜像规则,只需要在Dockerfile上添加或者修改指令, 就可生成docker 镜像.

1.2 DockerFile文件格式

Dockerfile 文件组成如下:

```
##  Dockerfile文件格式  # This dockerfile uses the ubuntu image# VERSION 2 -
EDITION 1# Author: docker_user# Command format: Instruction [arguments / command]
..  # 1、第一行必须指定 基础镜像信息FROM ubuntu  # 2、维护者信息MAINTAINER docker_user
docker_user@email.com  # 3、镜像操作指令RUN echo "deb
http://archive.ubuntu.com/ubuntu/ raring main universe" >>
/etc/apt/sources.listRUN apt-get update && apt-get install -y nginxRUN echo
"\ndaemon off;" >> /etc/nginx/nginx.conf  # 4、容器启动执行指令CMD /usr/sbin/nginx
```

Dockerfile 四部分说明:

- 一开始必须要指明所基于的镜像名称, 关键字是FROM, 这是必须的.
- 接下来是维护者信息关键字是MAINTAINER, 非必须, 但良好的习惯有利于后期的职责明确.
- 后面是镜像操作指令, 如RUN等, 每执行一条RUN命令,镜像添加新的一层.
- 最后是CMD指令,来指明运行容器时的操作命令.

2 和DockerFile说一声Hello

简单示例

在一个空白的文本文件, 命名为Dockerfile

```
mkdir mynginxcd mynginxvi Dockerfile
```

Dockerfile文件内容:

```
FROM nginxMAINTAINER WCC RUN echo '<h1>It works!</h1>' >
/usr/share/nginx/html/index.html
```

构建镜像

```
docker build -t nginx:v1 . Sending build context to Docker daemon  3.072kBStep
1/2 : FROM nginxlatest: Pulling from library/nginxbc51dd8edc1b: Pull
complete66ba67045f57: Pull completebf317aa10aa5: Pull completeDigest:
sha256:ad5552c786f128e389a0263104ae39f3d3c7895579d45ae716f528185b36bc6fStatus:
Downloaded newer image for nginx:latest ---> 2073e0bcb60eStep 2/2 : RUN echo
'<h1>It works!</h1>' > /usr/share/nginx/html/index.html ---> Running in
2f49ab8104beRemoving intermediate container 2f49ab8104be --->
e68df98a0494Successfully built e68df98a0494Successfully tagged nginx:v1
```

接下来是启动容器, 使用docker run 命令

```
docker run --rm  --name docker_nginx_v1 -d -p 80:80 nginx:v1
```

3 DockerFile中常见的13种指令

```
FROM        #基础镜像一切要从这里开始MAINTAINER # 镜像是谁写的   姓名+ 邮箱RUN         # 镜
像构建的时候要运行命令ADD         # tomcat 需要 压缩包   。 添加内容WORKDIR     # 镜像工作目
录VOLUME      # 挂载目录EXPOSE      # 保留端口配置CMD          # 指定容器启动的时候要运行命
令。 只有一个生效ENTRYPOINT # 指定容器启动的时候运行命令。 可以追加命令COPY         # 类似
ADD 将我们文件拷贝道镜像中ENV           # 构建的时候设置环境变量
```

FROM

> 指定基础镜像，必须为第一个命令

```
格式：    FROM <image>    FROM <image>:<tag>    FROM <image>@<digest>示例：
FROM mysql:5.6注：        tag或digest是可选的，如果不使用这两个值时，会使用latest版本的基础
镜像
```

MAINTAINER

> 维护者信息

```
格式：    MAINTAINER <name>示例：    MAINTAINER Jasper Xu    MAINTAINER
sorex@163.com    MAINTAINER Jasper Xu <sorex@163.com>
```

RUN

> 构建镜像时执行的命令

```
RUN用于在镜像容器中执行命令，其有以下两种命令执行方式：shell执行格式：    RUN <command>exec
执行格式：    RUN ["executable", "param1", "param2"]示例：    RUN ["executable",
"param1", "param2"]    RUN apk update    RUN ["/etc/execfile", "arg1", "arg1"]
注：    RUN指令创建的中间镜像会被缓存，并会在下次构建中使用。如果不想使用这些缓存镜像，可以在构
建时指定--no-cache参数，如：docker build --no-cache
```

ADD

> 将本地文件添加到容器中，tar类型文件会自动解压(网络压缩资源不会被解压)，可以访问网络资源，类似wget

```
格式：    ADD <src>... <dest>    ADD ["<src>",... "<dest>"] 用于支持包含空格的路径示
例：    ADD hom* /mydir/          # 添加所有以"hom"开头的文件    ADD hom?.txt /mydir/
    # ? 替代一个单字符,例如："home.txt"    ADD test relativeDir/    # 添加 "test"
到 `WORKDIR`/relativeDir/    ADD test /absoluteDir/    # 添加 "test" 到
/absoluteDir/        压缩格式为gzip、bzip2以及xz的情况下，ADD指令都会将其解压缩！非常值得
注意的是，目标路径为一个URL时，会将其自动下载到目标路径下，但是其权限被自动设置成了600，如果这并
不是你想要的权限，那么你还需要额外增加一层RUN命令进行更改，另外，如果下载的是一个压缩包，同样你
还需要额外增加一层RUN命令进行解压缩。所以，在这种情况下，你还不如指定只用一层RUN，使用curl或者
wget工具进行下载，并更改权限，然后进行解压缩，最后清理无用文件！当你的源路径为压缩文件并且不想让
Docker引擎将其自动解压缩，这个时候就不可以使用ADD命令，你可以使用COPY命令进行完成！其实ADD命令
并不实用，并不推荐使用！！！
```

COPY

> 功能类似ADD，但是是不会自动解压文件，也不能访问网络资源

```
COPY <源路径> ... <目标路径>原路径可以是多个，甚至是通配符，其通配规则只需要满足GO语言的
filepath.Math规则即可，如下：COPY ./test1.py ./test2.py /test/COPY ./t*.py
/test/COPY ./test?.py /test/目标路径是容器内的绝对路径，也可以是工作目录下的相对路径，工作
目录可以使用WORKDIR指令进行指定。目标路径不需要事先创建，Docker会自动创建所需的文件目录。使用
COPY指令会将源路径的文件的所有元数据，比如读、写、指定全选、时间变更等。如果源路径时一个目录，那
么会将整个目录复制到容器中，包括文件系统元数据。
```

CMD

> 构建容器后调用，也就是在容器启动时才进行调用。
>
> 指定这个容器启动的时候要运行的命令只有最后一个会生效 可被替换。

```
CMD指令与RUN指令相似，也具有两种格式：shell格式：CMD <命令>exec格式：CMD [“可执行文件”，“参数1”，“参数2”，…]之前介绍容器的时候就说过，Docker不是虚拟机，容器就是进程。既然是进程，那么在启动容器的时候，就需要指定运行的程序及参数。CMD就是指定默认的容器主进程的启动命令的。在运行时可以设置CMD指令来代替镜像设置中的命令，例如Ubuntu默认的CMD是/bin/bash，当我们使用命令docker run -it ubuntu创建并启动一个容器会直接进入bash。我们也可以在运行时指定运行别的命令，比如docker run -it ubuntu cat /etc/os-release，这就用cat /etc/os-release命令代替了默认的/bin/bash命令，输出了系统版本信息。比如，我想在启动容器的时候，在控制台中输出Hello Docker!，我们可以在Dockerfile中这样写，如下：FROM ubuntuCMD echo "Hello Docker!"接下来，我们构建一个镜像ubuntu:v1.0，接下来，我们以此镜像为基础创建并启动一个容器，如下：docker run -it ubuntu:v1.0值得注意的是，如果使用shell格式，那么实际的命令会被包装成为sh -c的参数的形式进行执行。上面的CMD指令，在实际执行中会变成：CMD ["sh", "-c", "echo", "Hello Docker!"]
```

ENTRYPOINT

> 配置容器，使其可执行化。配合CMD可省去"application"，只使用参数。
>
> 指定容器启动的要运行的命令，可以追加命令

```
ENTRYPOINT指令和CMD指令目的一样，都是指定容器运行程序及参数，并且与CMD一样拥有两种格式的写法：shell格式：ENTRYPOINT <命令>exec格式：ENTRYPOINT [“可执行文件”，“参数1”，“参数2”，…]与CMD指令一样，ENTRYPOINT也更加推荐使用exec格式，ENTRYPOINT在docker run命令中同样也可以进行指定，只不过比CMD指令来的繁琐一些，需要指定--entrypoint参数。同样，在docker run命令中指定了--entrypoint参数的话，会覆盖Dockerfile中ENTRYPOINT上的指令。当指定了ENTRYPOINT指令时，CMD指令里的命令性质将会发生改变！CMD指令中的内容将会以参数形式传递给ENTRYPOINT指令中的命令，如下：FROM ubuntuENTRYPOINT ["rm", "docker2"]CMD ["-rf"]其实，它真正执行的命令将会是：rm docker2 -rf
```

LABEL

> 用于为镜像添加元数据

```
格式：     LABEL <key>=<value> <key>=<value> <key>=<value> ...示例：     LABEL version="1.0" description="这是一个Web服务器" by="IT笔录"注：     使用LABEL指定元数据时，一条LABEL指定可以指定一或多条元数据，指定多条元数据时不同元数据之间通过空格分隔。推荐将所有的元数据通过一条LABEL指令指定，以免生成过多的中间镜像。
```

ENV

> 这个指令非常简单，就是用于设置环境变量而已，无论是接下来的指令，还是在容器中运行的程序，都可以使用这里定义的环境变量。例如：

```
FROM ubuntu:16.04ENV MODE=testRUN apt-get update && apt-get install -y curl && curl http://192.168.0.89:5000/$MODE && rm -rf /var/lib/apt/lists/*如果你要设置多个环境变量，为了美观，你可以使用\来进行换行。多个环境变量的隔开，使用空格进行隔开的，如果某个环境变量的值是由一组英文单词构成，那么你可以将其使用""进行圈起来。如下：FROM ubuntu:16.04RUN MODE=test DESCRITPION="ios 12" \    TITLE="iphone"        接下来，将这个Dockerfile构建成镜像，然后以此镜像为基础创建并启动一个容器，在容器中，我们调用这个环境变量，仍然是有用的！！！值得注意的是，如果你想通过CMD或者ENTRYPOINT指令的exec格式来打印环境，就像下面这样：CMD ["echo", $MODE]CMD ["echo", "$MODE"]这样都是不能正确输出环境变量的值的，你可以改成exec格式来执行shell命令，如下：CMD ["sh", "-c", "echo $MODE"]如此，就能正确输出环境变量的值了！
```

EXPOSE

> EXPOSE指令是声明运行时容器服务端口，这只是一个声明，在运行时并不会因为这个声明应用就会开启这个端口的服务。在Dockerfile中这样声明有两个好处：一个是帮助镜像使用者更好的理解这个镜像服务的守护端口，另一个作用则是在运行时使用随机端口映射时，也就是docker run -p命令时，会自动随机映射EXPOSE端口。
>
> 要将EXPOSE和在运行时使用-p <宿主>:<容器端口>区分开来，-p是映射宿主端口和容器端口，换句话说，就是将容器的对应端口服务公开给外界访问，而EXPOSE仅仅是声明端口使用什么端口而已，并不会自动在宿主进行端口映射。

```
格式：    EXPOSE <port> [<port>...]示例：    EXPOSE 80 443    EXPOSE 8080    EXPOSE 11211/tcp 11211/udp注：    EXPOSE并不会让容器的端口访问到主机。要使其可访问，需要在docker run运行容器时通过-p来发布这些端口，或通过-P参数来发布EXPOSE导出的所有端口
```

VOLUME

> `VOLUME`指令用于构建镜像时定义匿名卷，其格式有两种：

```
格式：    VOLUME ["/path/to/dir"]示例：    VOLUME ["/data"]    VOLUME ["/var/www", "/var/log/apache2", "/etc/apache2"注：    一个卷可以存在于一个或多个容器的指定目录，该目录可以绕过联合文件系统，并具有以下功能：1 卷可以容器间共享和重用2 容器并不一定要和其它容器共享卷3 修改卷后会立即生效4 对卷的修改不会对镜像产生影响5 卷会一直存在，直到没有任何容器在使用它FROM ubuntu:16.04VOLUME ["/data", "/command"]这里的/data和/command目录在容器运行时会自动挂载为匿名卷，任何向/data和/command目录中写入的信息都不会记录进容器存储层，从而保证了容器存储层的无状态化！容器匿名卷目录指定可以通过docker run命令中指定-v参数来进行覆盖
```

WORKDIR

> 工作目录，类似于cd命令

```
格式：    WORKDIR /path/to/workdir示例：    WORKDIR /a  (这时工作目录为/a)    WORKDIR b  (这时工作目录为/a/b)    WORKDIR c  (这时工作目录为/a/b/c)注：    通过WORKDIR设置工作目录后，Dockerfile中其后的命令RUN、CMD、ENTRYPOINT、ADD、COPY等命令都会在该目录下执行。在使用docker run运行容器时，可以通过-w参数覆盖构建时所设置的工作目录。案列FROM ubuntu:16.04WORKDIR /data/testRUN mkdir dockerWORKDIR /data/test/dockerRUN echo "test" > demo.txt这样，Dockerfile中两次RUN指令的操作都在不同的目录下进行，最终容器会切换到最后一次WORKDIR指令下的目录。WORKDIR指令可以通过docker run命令中的-w参数来进行覆盖
```

USER

> `USER`指令用于将会用以什么样的用户去运行

```
FROM ubuntu:16.04USER docker
```

基于该镜像启动的容器会以`docker`用户的身份来运行，我们可以指定用户名或者UID，组名或者GID，或者两者的结合，如下：

```
FROM ubuntu:16.04USER userUSER user:groupUSER uidUSER uid:gidUSER user:gidUSER
uid:group
```

`USER`指令可以在`docker run`命令中的`-u`参数进行覆盖

ARG

> 构建参数ARG和ENV指令一样，都是设置环境变量。与之不同的是，ARG设置的环境变量只是在镜像构建时所设置的，在将来容器运行时是不会存在这些环境变量的。但是不要因此就用ARG来保存密码之类的信息，因为通过docker history还是能够看得到的。ARG指令与ENV指令的使用类似

```
FROM ubuntu:16.04ARG app="python-pip"RUN apt-get update && apt-get install -y
$app && rm -rf /var/lib/apt/lists/*
```

`ARG`构建参数可以通过`docker run`命令中的`--build-arg`参数来进行覆盖

ONBUILD

> `ONBUILD`指令是为了帮助别人定制而准备的

```
格式：    ONBUILD [INSTRUCTION]示例：    ONBUILD ADD . /app/src    ONBUILD RUN
/usr/local/bin/python-build --dir /app/src注：    当所构建的镜像被用做其它镜像的基础镜
像，该镜像中的触发器将会被钥触发from ubuntu:16.04WORKDIR /dataONBUILD RUN mkdir
testdocker build -t test:v8 .此时，我们以此Dockerfile进行构建镜像ubuntu:test，并以此镜
像为基础创建并启动一个容器，进入容器后，容器会自动切换到WORKDIR指令下的目录，此时我们使用ls命令
会发现在工作目录下，并未创建test文件夹，如下：root@ubuntu:~/docker# docker run -it
test:v8root@3a8f912fd23b:/data# lsroot@3a8f912fd23b:/data#此时，我们再创建一个
Dockerfile，只需一个FROM指令即可，使其继承刚刚我们构建的ubuntu:test镜像，如下：FROM
test:v8我们再以此Dockerfile构建镜像ubuntu:test_onbuild，并以此镜像为基础创建并启动一个容
器，进入容器后，容器会自动切换到WORKDIR指令下的目录，此时我们使用ls命令会发现在工作目录下，已经
创建好了一个名为test的文件夹，如下：root@ubuntu:~/docker# docker run -it
ubuntu:test_onbuildroot@5394e605b6ea:/data# lstest
```

4 CMD和ENTRYPOINT的区别

```
CMD          #指定这个容器启动的时候要运行的命令，可被替换ENTRYPOINT #指定这个容器启动的时候要
运行的命令，可以追加命令
```

```
from centosCMD ["ls" ,"-a"]docker build -t test04:v1 .
```

5 DockerFile综合实验

5.1 DockerFile制作Tomcat镜像

dockerfile是用来构建dokcer镜像的文件!令参数脚本!构建步骤:

1. 编写一个dockerfile 文件
2. docker build构建成为一个镜像
3. docker run运行镜像

DockerFile

```
FROM centosMAINTAINER WCC<18587781058@163.com># 解压缩ADD ./apache-tomcat-9.0.50.tar.gz /usr/localADD ./jdk-8u201-linux-x64.tar.gz /usr/localRUN yum install -y vimENV MYPATH /usr/localWORKDIR $MYPATH          ENV JAVA_HOME=/usr/local/jdk1.8.0_201ENV TOMCAT_HOME=/usr/local/apache-tomcat-9.0.50ENV PATH $PATH:$JAVA_HOME/bin:$TOMCAT_HOME/bin:$TOMCAT_HOME/lib EXPOSE 8080             CMD /usr/local/apache-tomcat-9.0.50/bin/startup.sh && tail -F /usr/local/apache-tomcat-9.0.50/bin/logs/catatlina.out
```

五、 Docker实战之网络管理

1 Docker0网络详解

清空所有容器来测试

```
docker stop $(docker ps -aq)docker rm  $(docker ps -aq)
```

思考

> 每启动一个docker容器， docker就会给容器分配一个ip，只要安装docker就会有一个docker0 网卡
>
> 桥接技术 使用的 evth-pair技术。

evth-pair 就是一对的虚拟设备接口，它们都是成对出现，一段彼此连接。 正是因为这个特性，evth-parit桥梁。 连接各种各样网络设备

总结

```
docker 使用linux的桥接， 宿主机中时一个docker容器的网桥 。 docker0docker中所有网络接口都是虚拟的。 虚拟转发效率高只要容器删除 对应的网桥就没有了
```

2 容器互联-link

> 思考一个场景,我们编写了一个微服务，database url=ip:，项目不重启,数据库ip换掉了, 我们希望可以处理这个问题，可以名字来进行访问容器?

3 四种网络模式

> Docker使用Linux桥接，在宿主机虚拟一个Docker容器网桥(docker0)，Docker启动一个容器时根据Docker网桥的网段分配给容器一个IP地址，称为Container-IP，同时Docker网桥是每个容器的默认网关。因为在同一宿主机内的容器都接入同一个网桥，这样容器之间就能够通过容器的Container-IP直接通信。

四类网络模式

Docker网络模式	配置	说明
host模式	--net=host	容器和宿主机共享Network namespace。
container模式	--net=container:NAME_or_ID	容器和另外一个容器共享Network namespace。kubernetes中的pod就是多个容器共享一个Network namespace。
none模式	--net=none	容器有独立的Network namespace，但并没有对其进行任何网络设置，如分配veth pair 和网桥连接，配置IP等。
bridge模式	--net=bridge	（默认为该模式）

Host模式

如果启动容器的时候使用host模式，那么这个容器将不会获得一个独立的Network Namespace，而是和宿主机共用一个Network Namespace。容器将不会虚拟出自己的网卡，配置自己的IP等，而是使用宿主机的IP和端口。但是，容器的其他方面，如文件系统、进程列表等还是和宿主机隔离的。

使用host模式的容器可以直接使用宿主机的IP地址与外界通信，容器内部的服务端口也可以使用宿主机的端口，不需要进行NAT，host最大的优势就是网络性能比较好，但是docker host上已经使用的端口就不能再用了，网络的隔离性不好。

Container模式

这个模式指定新创建的容器和已经存在的一个容器共享一个 Network Namespace，而不是和宿主机共享。新创建的容器不会创建自己的网卡，配置自己的 IP，而是和一个指定的容器共享 IP、端口范围等。同样，两个容器除了网络方面，其他的如文件系统、进程列表等还是隔离的。两个容器的进程可以通过 lo 网卡设备通信。

None模式

使用none模式，Docker容器拥有自己的Network Namespace，但是，并不为Docker容器进行任何网络配置。也就是说，这个Docker容器没有网卡、IP、路由等信息。需要我们自己为Docker容器添加网卡、配置IP等。

这种网络模式下容器只有lo回环网络，没有其他网卡。none模式可以在容器创建时通过--network=none来指定。这种类型的网络没有办法联网，封闭的网络能很好的保证容器的安全性。

Bridge模式

当Docker进程启动时，会在主机上创建一个名为docker0的虚拟网桥，此主机上启动的Docker容器会连接到这个虚拟网桥上。虚拟网桥的工作方式和物理交换机类似，这样主机上的所有容器就通过交换机连在了一个二层网络中。

从docker0子网中分配一个IP给容器使用，并设置docker0的IP地址为容器的默认网关。在主机上创建一对虚拟网卡veth pair设备，Docker将veth pair设备的一端放在新创建的容器中，并命名为eth0（容器的网卡），另一端放在主机中，以vethxxx这样类似的名字命名，并将这个网络设备加入到docker0网桥中。可以通过brctl show命令查看。

bridge模式是docker的默认网络模式，不写--net参数，就是bridge模式。使用docker run -p时，docker实际是在iptables做了DNAT规则，实现端口转发功能。可以使用iptables -t nat -vnL查看。

4 自定义网络

建议使用自定义的网桥来控制哪些容器可以相互通信，**还可以自动DNS解析容器名称到IP地址。**

基础命令

```
docker network --helpconnect 将某个容器连接到一个docker网络create 创建一个docker局域网络disconnect 将某个容器退出某个局域网络inspect 显示某个局域网络信息ls 显示所有docker局域网络prune 删除所有未引用的docker局域网络rm 删除docker网络docker network ls
    #查看网络连接docker network inspect <network>    #查看网络连接详情docker network connect <network> <container>           #容器连接网络docker network disconnect [-f] <network> <container>   #断开网络连接docker network create <network>   #创建网络docker network rm <network>       #删除网络连接docker network prune              #删除没有使用的网络连接   删除前会有确认
```

自定义网络

```
    docker network create --driver bridge --subnet 192.168.0.0/16 --gateway 192.168.0.1 mynet##bridge -网络模式桥连接 subnet - 子网络网段 gateway - 路由 mynet- 网络名查看网络信息：docker network inspect mynet我们在自己的网段里创建两个容器docker run -d -P --name tomcat01 --net mynet tomcatdocker run -d -P --name tomcat02 --net mynet tomcatdocker exec -it tomcat01 ping tomcat02查看自定义网络信息docker network inspect mynet
```

5 网络连接

```
[root@docker ~]# docker network connect --helpUsage:  docker network connect [OPTIONS] NETWORK CONTAINERConnect a container to a networkOptions:      --alias strings           Add network-scoped alias for the container      --driver-opt strings      driver options for the network      --ip string               IPv4 address (e.g., 172.30.100.104)      --ip6 string              IPv6 address (e.g., 2001:db8::33)      --link list               Add link to another container      --link-local-ip strings   Add a link-local address for the container
```

思考: tomcat01和Mysql01是否可以ping通?

```
docker network connect mynet tomcat01
```

6 网络综合实战

6.1 Redis集群部署实战

6.1.1 设置redis集群网卡及查看

```
docker network create redis --subnet 172.38.0.0/16docker network lsdocker network inspect redis
```

6.1.2 redis节点创建及配置

```
for port in $(seq 1 6);domkdir -p /mydata/redis/node-${port}/conftouch
/mydata/redis/node-${port}/conf/redis.confcat << EOF
>/mydata/redis/node-${port}/conf/redis.confport 6379bind 0.0.0.0cluster-enabled
yescluster-config-file nodes.confcluster-node-timeout 5000cluster-announce-ip
172.38.0.1${port}cluster-announce-port 6379cluster-announce-bus-port
16379appendonly yesEOFdone
```

6.1.3 拉取redis镜像并启动redis节点

```
//节点1docker run -p 6371:6379 -p 16371:16379 --name redis-1 \ -v
/mydata/redis/node-1/data:/data \ -v /mydata/redis/node-
1/conf/redis.conf:/etc/redis/redis.conf \ -d --net redis --ip 172.38.0.11
redis:5.0.9-alpine3.11 redis-server /etc/redis/redis.conf//节点2docker run -p
6372:6379 -p 16372:16379 --name redis-2 \ -v /mydata/redis/node-2/data:/data \ -
v /mydata/redis/node-2/conf/redis.conf:/etc/redis/redis.conf \ -d --net redis --
ip 172.38.0.12 redis:5.0.9-alpine3.11 redis-server /etc/redis/redis.conf//节点
3docker run -p 6373:6379 -p 16373:16379 --name redis-3 \ -v /mydata/redis/node-
3/data:/data \ -v /mydata/redis/node-3/conf/redis.conf:/etc/redis/redis.conf \ -
d --net redis --ip 172.38.0.13 redis:5.0.9-alpine3.11 redis-server
/etc/redis/redis.conf//节点4docker run -p 6374:6379 -p 16374:16379 --name redis-4
\ -v /mydata/redis/node-4data:/data \ -v /mydata/redis/node-
4/conf/redis.conf:/etc/redis/redis.conf \ -d --net redis --ip 172.38.0.14
redis:5.0.9-alpine3.11 redis-server /etc/redis/redis.conf//节点5docker run -p
6375:6379 -p 16375:16379 --name redis-5 \ -v /mydata/redis/node-5/data:/data \ -
v /mydata/redis/node-5/conf/redis.conf:/etc/redis/redis.conf \ -d --net redis --
ip 172.38.0.15 redis:5.0.9-alpine3.11 redis-server /etc/redis/redis.conf//节点
6docker run -p 6376:6379 -p 16376:16379 --name redis-6 \ -v /mydata/redis/node-
6/data:/data \ -v /mydata/redis/node-6/conf/redis.conf:/etc/redis/redis.conf \ -
d --net redis --ip 172.38.0.16 redis:5.0.9-alpine3.11 redis-server
/etc/redis/redis.conf
```

6.1.4 生成集群:

```
进入容器docker exec -it redisdocker exec -it redis-1 /bin/shredis-cli --cluster
create 172.38.0.11:6379 172.38.0.12:6379 172.38.0.13:6379 172.38.0.14:6379
172.38.0.15:6379 172.38.0.16:6379 --cluster-replicas 1
```

6.1.5 查看集群状态

```
#连接客户端redis-cli -c # 查看集群信息cluster info# 查看集群节点cluster nodes
```

六、Docker实战之部署微服务

6.1 创建SpringBoot应用

```
@RestControllerpublic class HelloController {    @GetMapping("/hello")    public
String success(){        return "hello baizhan";    }}
```

6.2 编写Dockerfile

```
FROM java:8COPY dockertest-0.0.1-SNAPSHOT.jar /app.jarCMD ["--server.port=8080"]EXPOSE 8080ENTRYPOINT ["java","-jar","/app.jar"]
```

6.3 构建微服务镜像

```
docker build -t springboot:v1 .
```

6.4 启动容器

```
docker run --name myspringboot -d -p 8080:8080 springboot:v1
```

七、Docker镜像的创建管理和发布

7.1 发布镜像到DockerHub

> DockerHub https://hub.docker.com/signup

1. 地址注册自己账号
2. 确定这个账号可以登录
3. 在我们服务器上提交自己的镜像

发布基本命令

```
#帮助docker login --help#登录docker login -u wcc#提交镜像docker push wcc/springboot:v1 [root@docker opt]# docker push springboot:v1The push refers to repository [docker.io/library/springboot]015fa7f968eb: Preparing 35c20f26d188: Preparing c3fe59dd9556: Preparing 6ed1a81ba5b6: Preparing a3483ce177ce: Preparing ce6c8756685b: Waiting 30339f20ced0: Waiting 0eb22bfb707d: Waiting a2ae92ffcd29: Waiting
```

7.2 发布镜像到阿里云

1 登录阿里云Docker Registry

```
$ docker login --username=1858778****@163.com registry.cn-hangzhou.aliyuncs.com用于登录的用户名为阿里云账号全名，密码为开通服务时设置的密码。您可以在访问凭证页面修改凭证密码。
```

3 将镜像推送到Registry

```
$ docker login --username=1858778****@163.com registry.cn-hangzhou.aliyuncs.com$ docker tag [ImageId] registry.cn-hangzhou.aliyuncs.com/kalista/mytomcat:[镜像版本号]$ docker push registry.cn-hangzhou.aliyuncs.com/kalista/mytomcat:[镜像版本号]
```

2 从Registry中拉取镜像

```
$ docker pull registry.cn-hangzhou.aliyuncs.com/kalista/mytomcat:[镜像版本号]
```

八、Docker环境下的企业级前后端分离项目部署

8.1 项目介绍

环境参数

- 操作系统 CentOS 8
- 容器 Docker 20.10.7
- 中间件 Nginx 1.20
- 后台环境 JDK1.8

项目部署架构图

8.2 项目演示

8.3 项目下载与配置

> 人人开源 https://www.renren.io/

8.4 编译后端代码

8.5 编译前端代码

8.5.1 安装Node.js

```
https://nodejs.org/en/
```

8.5.2 安装cnpm淘宝镜像

> *CNPM* 是中国 npm 镜像的客户端。

安装cnpm

```
npm install -g cnpm --registry=https://registry.npm.taobao.org
```

8.5.3 编译前端代码

1. 利用Webpack打包工程
2. 执行webpack打包命令

```
cnpm run build
```

2. 打包之后的文件需要拷贝到Linux系统中

```
使用xftp
```

8.6 安装Redis集群高可用

参考 2.6

8.7 安装Mysql服务

```
docker run -d -p 3306:3306 \-v /opt/mysql/conf:/etc/mysql/conf.d  \-v
/opt/mysql/data:/var/lib/mysql \-e MYSQL_ROOT_PASSWORD=123456 --name mysql
mysql:8.0
```

8.8 服务搭建架构说明

8.9 后台服务部署容器

8.9.1 查看网络架构图

8.9.2 后台服务容器化

编写Dockerfile

```
FROM java:8  COPY renren-fast.jar  /app.jarCMD ["--server.port=8080"]EXPOSE
8080ENTRYPOINT ["java","-jar","/app.jar"]
```

启动容器

```
docker run --name r1  -d -p  6200:8080  renren-service:v1docker run --name r2 -d
-p  6201:8080   renren-service:v1docker run --name r3 -d -p  6202:8080   renren-
service:v1加入 redis 网卡docker network connect redis r1docker network connect
redis r2docker network connect redis r3
```

8.10 配置后台服务负载均衡

容器中Nginx位置

```
日志文件位置:   /var/log/nginx配置文件位置:   /etc/nginx资源存放的位置:
/usr/share/nginx/html
```

拷贝文件初始文件志宿主机中

```
docker cp 容器id:/etc/nginx /opt/nginxdocker run --name test -d -p 80:80 \-v
/opt/nginx:/etc/nginx \-d nginx
```

负载均衡配置

首先在宿主机nginx目录中修改nginx.conf

```
   upstream tomcat {      server 192.168.66.100:6200;      server
192.168.66.100:6201;      server 192.168.66.100:6202;    }
```

首先在宿主机conf.d修改 default.conf

```
  location / {        proxy_pass http://tomcat;        index  index.html
index.htm;    }
```

8.11 前台项目容器化

8.11.1 修改vue项目中的全局ip

```
/static/config/index.js
```

8.11.2 创建前端数据卷挂载目录

```
docker volume create renren-nginx
```

8.11.3 前台项目容器化

```
docker run --name n1 -d -p 6300:80 -v renren-nginx:/usr/share/nginx/html
nginxdocker run --name n2 -d -p 6301:80 -v renren-nginx:/usr/share/nginx/html
nginxdocker run --name n3 -d -p 6302:80 -v renren-nginx:/usr/share/nginx/html
nginx
```

8.12 前台项目负载均衡

> 参考8.10